

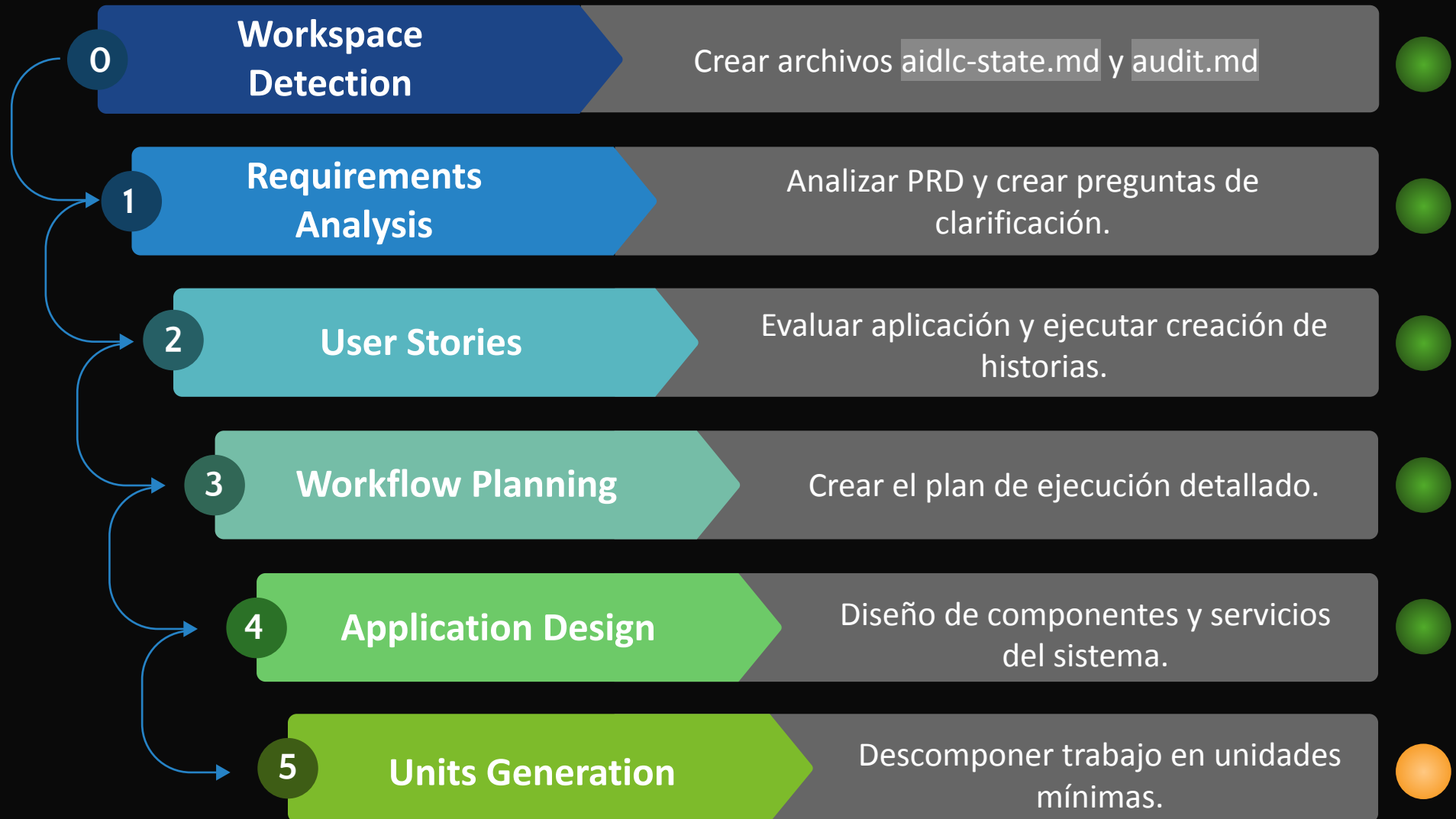
Estación 5

Diseñando el CÓMO

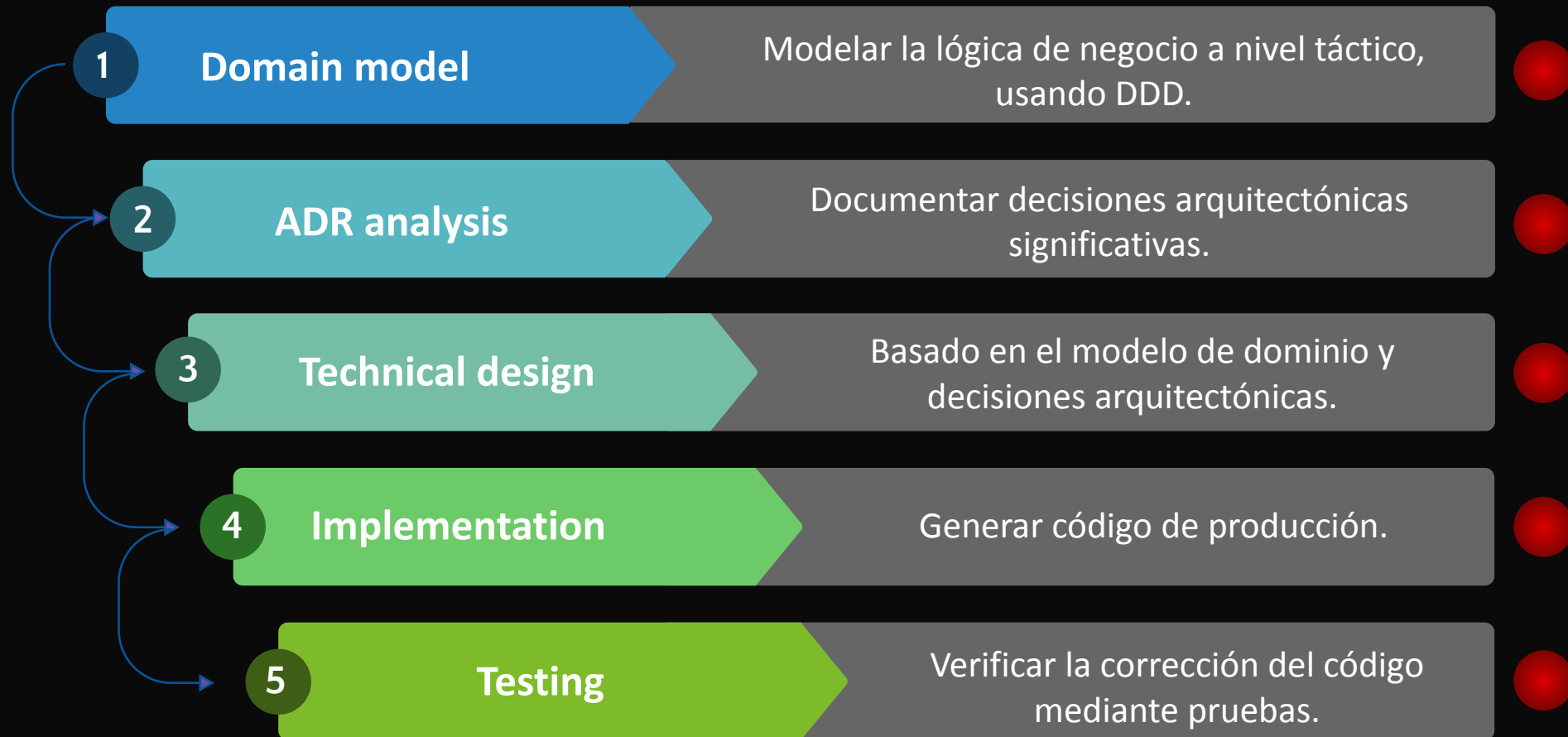
Construction con AI-DLC - DDD - ADR - TDD

Christian Braatz · Hardcore AI | 30X

¿Qué hemos construido hasta ahora? - Inception



¿Qué construiremos hoy? - Construction



Módulo 01

AI-DLC: Etapas de la construcción

Fase Construction - Etapas

- Confirmar contextos
- Eventos
- Un solo lenguaje

Modelo de
dominio



Diseño
técnico



- Patrones
- Interfaces
- Estructuras

- Opciones
- Justificaciones
- Consecuencias

Análisis
ADR



Implementación y
Pruebas

- Patrones y estándares
- Código mantenible
- TDD, Integración y aceptación

Módulo 02

Hands-On AI-DLC - Fase2: Construction (El Cómo)

Objetivo: Transformar las especificaciones en código funcional y probado a través de etapas disciplinadas.

Re-enfoquemos el framework

¿Cómo podemos retomar el framework para avanzar a nuestro ritmo? ...



PROMPT / CONTEXTO

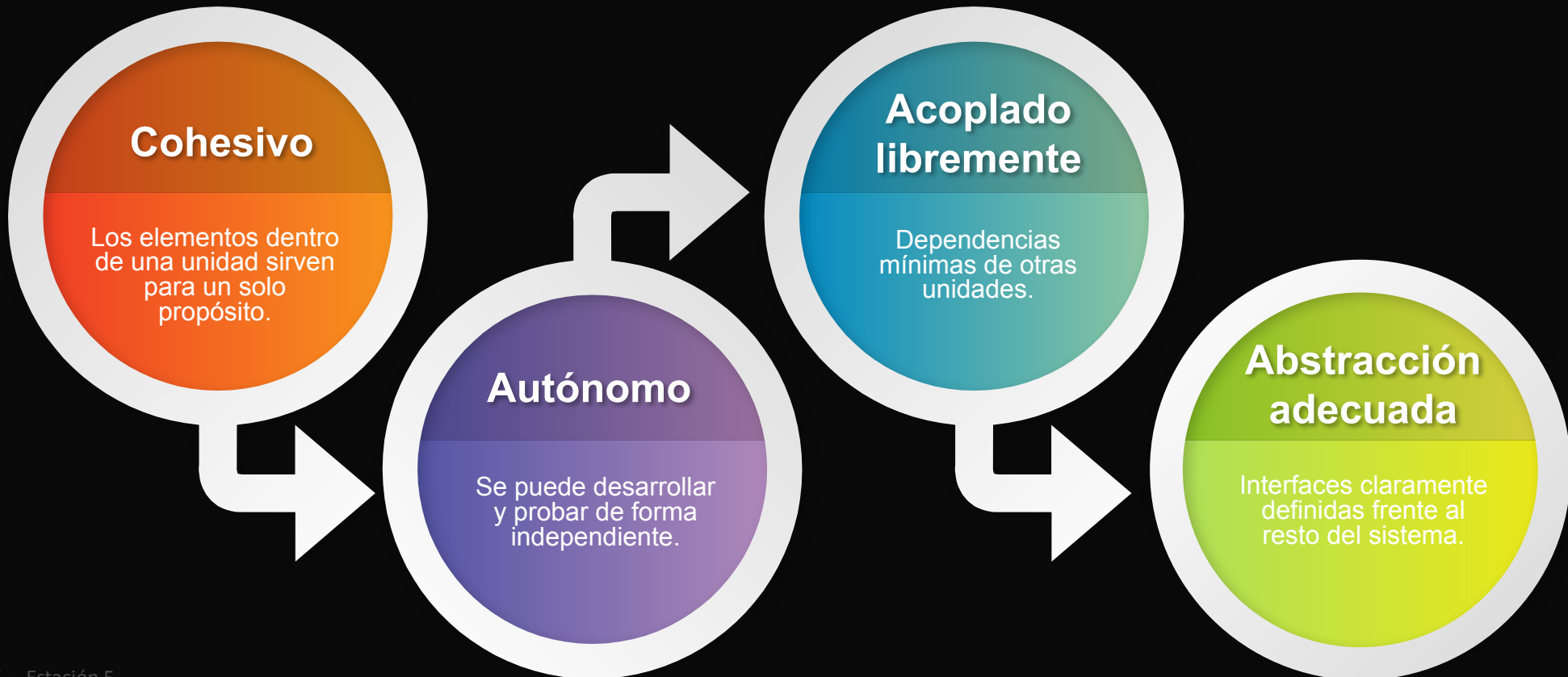
Confírmame en qué fase de AI-DLC nos encontramos, para avanzar.

Recordemos...

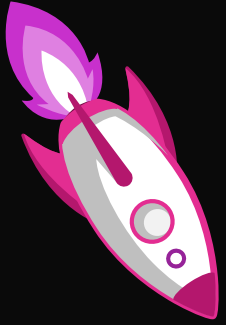
¿Qué es una Unidad de Trabajo

Una **Unidad** es un elemento de trabajo cohesivo y autónomo derivado de una Intención.

Las unidades están acopladas libremente y se pueden desarrollar de forma independiente.



El viaje de una Unidad de Trabajo



Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas

Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código

Actividad 1: Diseño funcional

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas

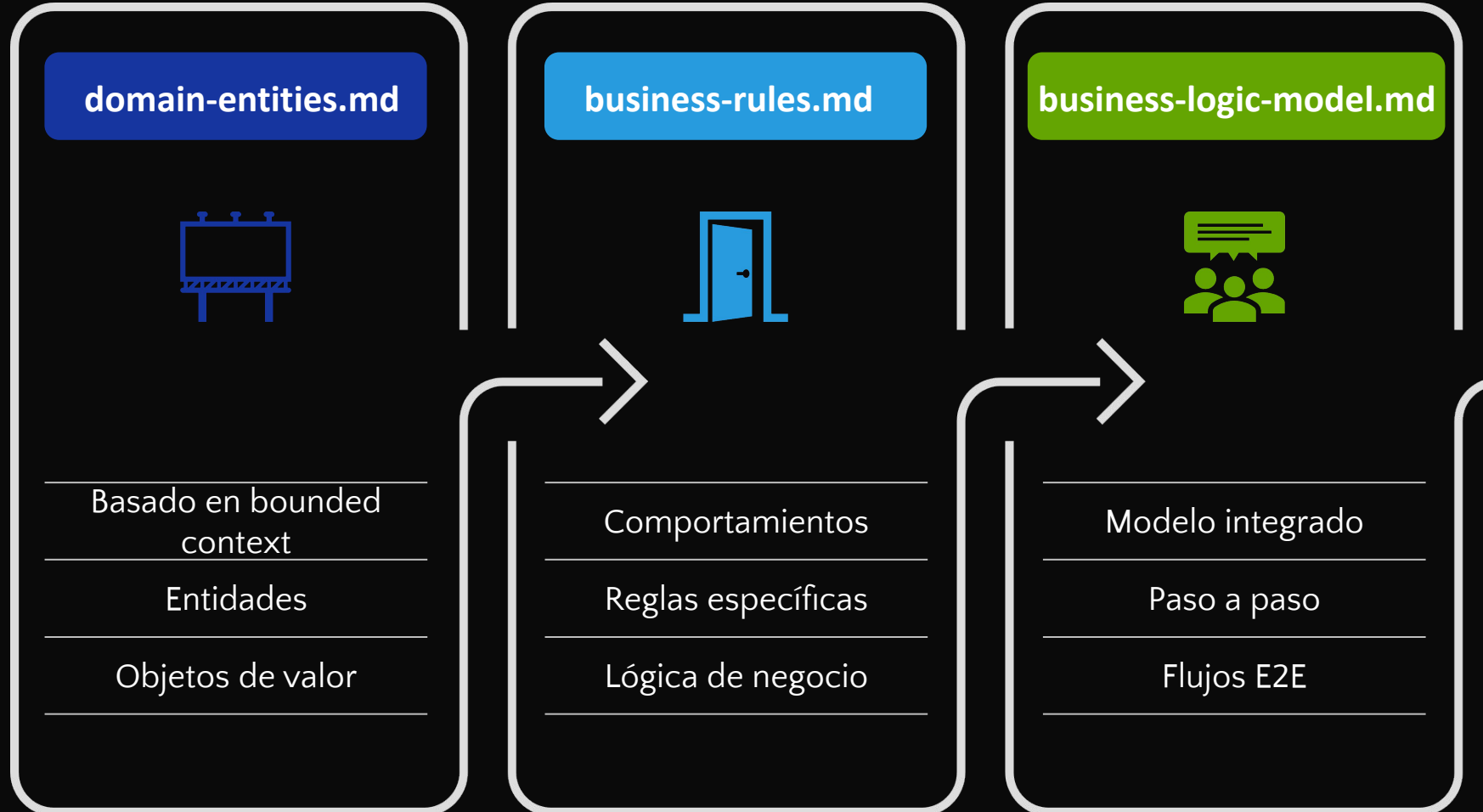


Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código

Actividad 1: Diseño funcional



Actividad 2: Requerimientos no funcionales

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas



Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código

Actividad 2: Requerimientos no funcionales

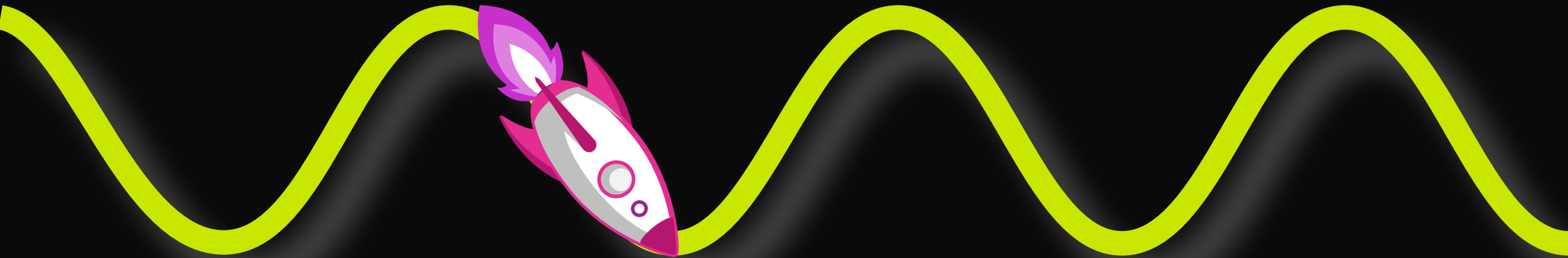
					
Desempeño	Seguridad	Confiabilidad	Usabilidad	Mantenibilidad	Escalabilidad
Especifican la eficiencia con la que debe funcionar el sistema. Ejemplo: "El sistema debe responder a las consultas de los usuarios en un plazo de 2 segundos en condiciones de carga normal y en un plazo de 4 segundos en condiciones de carga máxima (definida como 10 000 usuarios simultáneos)".	Definen cómo el sistema protege los datos y la funcionalidad del acceso no autorizado y posibles amenazas. Ejemplo: "Todas las contraseñas de usuario deben almacenarse utilizando el algoritmo hash bcrypt con un factor de trabajo mínimo de 10, y todas las transmisiones de datos deben utilizar cifrado TLS 1.3 o superior".	Especifican la capacidad del sistema para funcionar de manera consistente en las condiciones establecidas. Ejemplo: "El sistema debe mantener una disponibilidad del 99,9% (no más de 8,76 horas de inactividad al año), sin que ninguna interrupción supere los 10 minutos".	Especifican qué tan fácil debe ser aprender a usar, utilizar e interactuar con el sistema. Ejemplo: "Los usuarios que lo utilizan por primera vez deberían poder completar el proceso de registro sin ayuda en menos de 3 minutos, con una tasa de éxito de la tarea de al menos el 95 %".	Especifican la facilidad con la que se puede modificar, mejorar o reparar el sistema. Ejemplo: "Cualquier componente del sistema debe ser modificable sin requerir cambios en más de otros dos componentes, y todas las modificaciones deben ser verificables mediante pruebas automatizadas".	Especifican qué tan bien puede adaptarse el sistema a mayores demandas. Ejemplo: "El sistema debe soportar un aumento del 200% en la base de usuarios sin requerir cambios arquitectónicos y con no más de un 20% de degradación en el tiempo de respuesta".

Actividad 3: Diseño de requerimientos no funcionales

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas



Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código

Actividad 3: Diseño de requerimientos no funcionales

Pregunta (cómo)

01



Decisión de Disponibilidad

Q6: ¿Qué nivel de disponibilidad se requiere para el MVP?

Selección: A) 99.5% mensual (~3.6h downtime/mes).
Balance óptimo costo/estabilidad para MVP.

NF Req.

02



NFR-U1-03 (Disponibilidad)

Métrica objetivo: **99.5%**.
Implementación de modos degradados para resiliencia.

Diseño NFR

03



Patrón 1: Degraded Mode (Redis → PostgreSQL Fallback)

- **Problema:** El fallo de Redis bloquea SessionStore, interrumpiendo al candidato.

- **Solución:** Bloque Try/Except con fallback transparente a PostgreSQL para persistencia de sesión.

- **Impacto:** Garantiza el cumplimiento de **NFR-U1-03** ante fallos de infraestructura de caché.

Kahoot

SDD - AI-DLC

Actividad 4: Diseño de infraestructura

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas

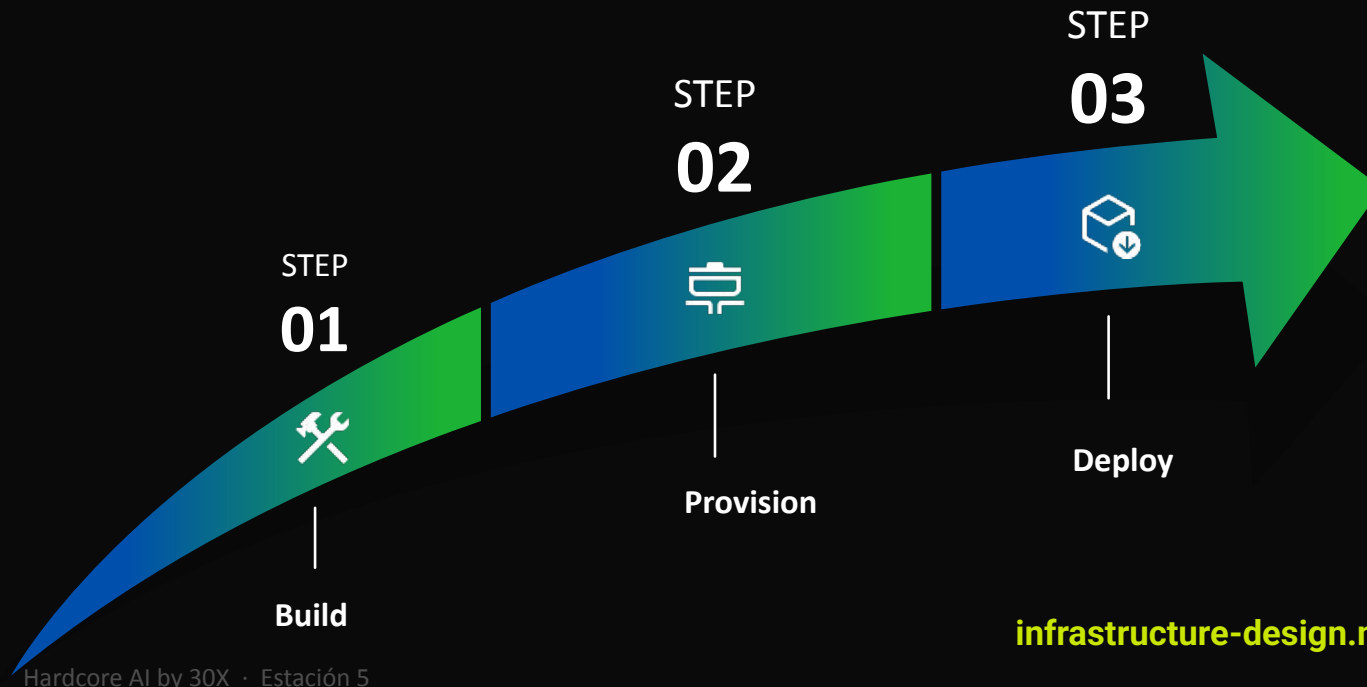
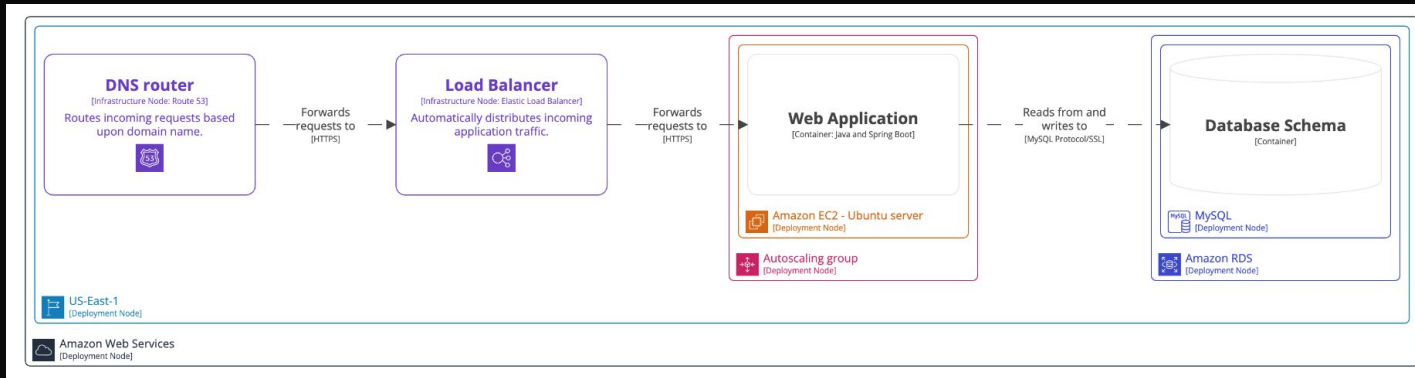


Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código

Actividad 4: Diseño de infraestructura



Mapa de servicios de infraestructura

Componente, servicio y configuración base



Configuración por servicio

Recursos, arquitectura, métodos, políticas, y cualquier otro asociado al servicio



Definiciones transversales

Seguridad, observabilidad (logs). distribución de redes.



Actividad 5: Generación de código

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas

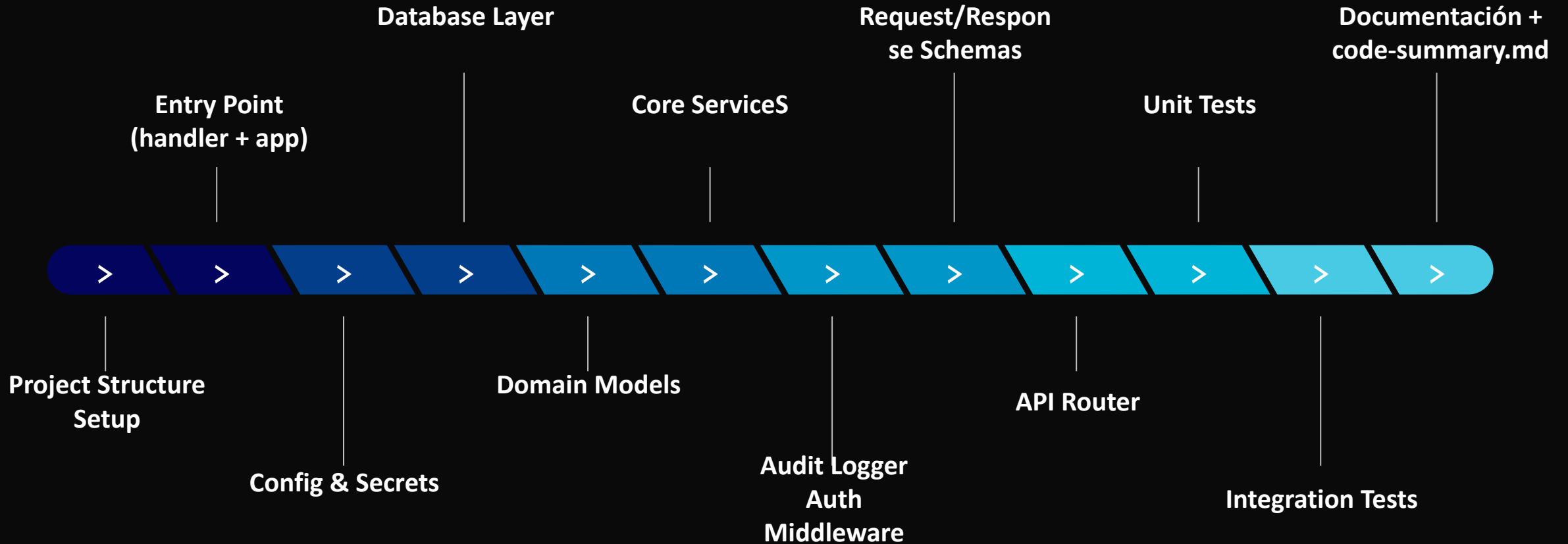
Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código



Actividad 5: Generación de código



Actividad 6: Pruebas - Diseño

Requerimientos
no funcionales

Diseño de
infraestructura

Pruebas

Diseño funcional

Diseño de requerimientos
no funcionales

Generación de
código



Actividad 6: Pruebas - Diseño



**Alta Testabilidad
Orgánica**



**Pruebas de
Comportamiento
vs. Pruebas de
Implementación**



**Cobertura de
Casos Borde (Edge
Cases)**



**Código Mínimo
Indispensable**

```
✓ tests
  ✓ integration
    __init__.py
    test_auth_endpoints.py
    test_operator_endpoints.py
  ✓ unit
    __init__.py
    test_auth_service.py
    test_brute_force.py
    test_operator_manager.py
    __init__.py
```

Actividad 6: Pruebas - Diseño

```
describe('InterviewEvaluatorService - AAA', () => {
  it('debería aprobar al candidato si su puntaje supera el umbral mínimo', () => {
    // Arrange: Preparamos los objetos y el estado inicial
    const minPassingScore = 75;
    const evaluator = new InterviewEvaluatorService(minPassingScore);
    const candidateScore = 85;
    const candidate = new Candidate("Juan Pérez", candidateScore);

    // Act: Ejecutamos el método que queremos probar
    const result = evaluator.evaluateCandidate(candidate);

    // Assert: Verificamos el estado o valor resultante
    expect(result.isApproved).toBe(true);
    expect(result.status).toBe('PROCEEDS_TO_NEXT_ROUND');
  });
});
```

Enfoque: Arrange - Act - Assert (AAA)

Cuando estemos haciendo pruebas unitarias muy puras de clases utilitarias, algoritmos matemáticos o transformaciones de datos donde el enfoque es estrictamente algorítmico (ej. un *retry pattern* o un formateador de fechas).

```
describe('InterviewEvaluatorService - GWT', () => {
  it('aprueba a un candidato con un puntaje superior al requerido', () => {
    // Given: Dado un contexto o estado del mundo (Regla de negocio)
    const aRequiredScoreOf = 75;
    const theInterviewEvaluator = new InterviewEvaluatorService(aRequiredScoreOf);
    const aCandidateWithScoreOf = 85;
    const theCandidate = new Candidate("Juan Pérez", aCandidateWithScoreOf);

    // When: Cuando ocurre un evento o acción de negocio
    const screeningResult = theInterviewEvaluator.evaluateCandidate(theCandidate);

    // Then: Entonces esperamos este resultado observable
    expect(screeningResult.isApproved).toBe(true);
    expect(screeningResult.status).toBe('PROCEEDS_TO_NEXT_ROUND');
  });
});
```

Enfoque: Given - When - Then (GWT)

Cuando estemos probando la lógica del dominio, casos de uso, o servicios de aplicación. Al escribir Given, When, Then, nos forzamos a pensar en **comportamientos** (lo que al negocio le importa) en lugar de en la **implementación** (lo que al compilador le importa).

Actividad

Para la próxima estación...

Previo a la estación 6

Aplica Construction completo a 2+ unidades de tu producto

- ▶ ☐ **Diseño funcional** por unidad: `domain-entities.md` + `business-rules.md` + `business-logic-model.md`
- ▶ ☐ **Análisis NFR** con valores numéricos concretos para los 6 atributos relevantes (por unidad)
- ▶ ☐ **ADR Design** — al menos **1 ADR documentado por unidad** (Contexto → Decisión → Consecuencias  )
- ▶ ☐ **Diseño de infraestructura** — `infrastructure-design.md` + `deployment-architecture.md` con diagrama Mermaid



El enfoque NO es llegar al código. Es generar las specs y decisiones de arquitectura más completas posibles. El código es consecuencia — la spec es el objetivo.